

Московский авиационный институт
(национальный исследовательский университет)

Факультет Компьютерные науки и прикладная математика

Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу «Дискретный анализ»

Студент: Савельев Артем
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 30 апреля 2026 г.
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №4

Условие задачи:

Требуется разработать программу для поиска заданной последовательности слов (паттерна) в тексте, представленном в виде последовательности слов, прочитанном из стандартного потока ввода с учетом информации о строках и позициях.

Вариант алгоритма: Алгоритм поиска подстроки Кнута-Морриса-Пратта (КМП с использованием Z-algorithm).

Входные данные: Первая строка содержит паттерн (последовательность слов до 16 символов); последующие строки содержат текст для поиска.

Выходные данные: Для каждого вхождения паттерна выводится номер строки и позиция первого слова паттерна на этой строке в формате «строка, позиция».

1 Описание

Требуется реализовать алгоритм поиска подстроки в последовательности элементов (слов в тексте) используя алгоритм Кнута-Морриса-Пратта (КМП). Особенностью этой реализации является построение функции отказов через промежуточное вычисление Z-array (массива префиксов из Z-algorithm), а затем конвертация его в так называемый «strong prefix function» для оптимизированного поиска.

1.1 Основная идея

Алгоритм КМП решает задачу поиска паттерна длины m в тексте длины n за время $O(n+m)$ без предварительной обработки всего текста в памяти. Вместо наивного перебора со сдвигом на каждый символ, КМП использует информацию о структуре паттерна, чтобы пропускать заведомо неудачные сдвиги.

1.2 Z-algorithm и Strong Prefix Function

Z-array массива p представляет собой массив длин наибольших префиксов, совпадающих с подстроками, начинающимися на каждой позиции:

$$z[i] = \max\{k : p[0..k-1] = p[i..i+k-1]\}$$

Strong prefix function преобразует Z-array так, что для каждой позиции j в паттерне хранится максимальная длина суффикса, который совпадает с префиксом и заканчивается именно в позиции j .

Построение strong prefix function из Z-array:

$$sp[i + z[i] - 1] = \max(sp[i + z[i] - 1], z[i]) \quad \text{для всех } i$$

2 Исходный код

Реализация разделена на модули: `utils.hpp/utils.cpp` содержат утилиты для построения структур данных и поиска, а `lab4.cpp` содержит главную программу.

2.1 Построение Z-array

Алгоритм построения Z-array работает за линейное время $O(m)$ и использует технику скользящего окна:

```
1 std::vector<int> build_z(const std::vector<std::string>& p) {
2     int m = (int)p.size();
3     std::vector<int> z(m, 0);
4     for (int i = 1, l = 0, r = 0; i < m; ++i) {
5         if (i <= r) {
6             z[i] = std::min(r - i + 1, z[i - 1]);
7         }
8         while (i + z[i] < m && p[z[i]] == p[i + z[i]]) {
9             z[i]++;
10        }
11        if (i + z[i] - 1 > r) {
12            l = i;
13            r = i + z[i] - 1;
14        }
15    }
16    return z;
17 }
```

Переменные l и r отслеживают границы наиболее правого отрезка, для которого известна информация о совпадении с префиксом. Это позволяет избежать избыточного сравнения и достичь линейности.

2.2 Построение Strong Prefix Function

```
1 std::vector<int> build_strong_pi(const std::vector<int>& z) {
2     int m = (int)z.size();
3     std::vector<int> sp(m, 0);
4
5     for (int i = m - 1; i >= 0; --i) {
6         if (z[i] > 0) {
7             int end_idx = i + z[i] - 1;
8             sp[end_idx] = std::max(sp[end_idx], z[i]);
9         }
10    }
11    return sp;
12 }
```

2.3 Поиск в потоке

```
1 void kmp_stream_search(std::istream& in,
2                        const std::vector<std::string>& pattern,
3                        const std::vector<int>& sp) {
4     if (pattern.empty()) return;
```

```

5
6     const int m = (int)pattern.size();
7     std::vector<Position> buffer(m);
8
9     std::string line;
10    size_t line_num = 1;
11    size_t total_words = 0;
12    int j = 0;
13
14    std::string current_word;
15    current_word.reserve(100);
16
17    while (std::getline(in, line)) {
18        size_t word_idx = 1;
19
20        for (size_t i = 0; i <= line.length(); ++i) {
21            if (i == line.length() ||
22                std::isspace((unsigned char)line[i])) {
23                if (!current_word.empty()) {
24                    to_lower_inplace(current_word);
25
26                    buffer[total_words % m] = {line_num, word_idx};
27                    ++total_words;
28
29                    while (j > 0 && pattern[j] != current_word) {
30                        j = sp[j - 1];
31                    }
32                    if (pattern[j] == current_word) {
33                        ++j;
34                    }
35
36                    if (j == m) {
37                        const auto& start_pos =
38                            buffer[(total_words - m) % m];
39                        std::cout << start_pos.line << ", "
40                            << start_pos.word_idx << "\n";
41                        j = sp[m - 1];
42                    }
43
44                    current_word.clear();
45                    word_idx++;
46                }
47                else {
48                    current_word += line[i];
49                }
50            }
51            line_num++;
52        }
53    }

```

Ключевая оптимизация: используется циклический буфер размером m , чтобы хранить только последние m позиций слов. Это обеспечивает $O(m)$ использования памяти вместо хранения всех позиций.

Основной цикл КМР выполняет следующие операции:

1. Читает строку из потока и парсит её на слова
2. Для каждого слова выполняет стандартный КМР-шаг: backtracking при несовпадении через функцию отказов
3. При полном совпадении паттерна выводит позицию и переходит к следующему возможному совпадению

Сложность алгоритма: предобработка паттерна за $2m$, следовательно асимптотически $O(n + m)$, где n — общее число букв в словах текста, m — число букв в словах паттерна.

3 Тест производительности

Тест производительности сравнивает время работы реализованного алгоритма КМП с использованием Z-algorithm и strong prefix function с библиотечной функцией `std::search` из STL C++. Хотя `std::search` может быть оптимизирована на уровне компилятора и использовать продвинутые эвристики (например, алгоритм Бойера-Мура), это сравнение показывает практическую производительность нашей реализации.

Тестовые данные (примерно 575 000 слов) были сгенерированы Python-скриптом `test_generator`. Паттерн состоит из нескольких слов. Время измерялось в микросекундах (`us`) с использованием библиотеки `<chrono>`.

3.1 Результаты базовых бенчмарков

```
1 ./benchmark < test_input.txt
2 Text size: 574814 words
3 Pattern size: 4 words
4 -----
5 Custom KMP: 2125 us (Found: 4911)
6 STL std::search: 1309 us (Found: 4911)
7 -----
8 Speedup: 0.616x
9
10 ./benchmark < test_input.txt
11 Text size: 2963615 words
12 Pattern size: 100 words
13 -----
14 Custom KMP: 26869 us (Found: 5010)
15 STL std::search: 12909 us (Found: 5010)
16 -----
17 Speedup: 0.680442x
```

3.2 KILLER_TEST: патологический случай

Для демонстрации истинного преимущества алгоритма КМП над наивным поиском был создан специальный тест, который генерируется скриптом:

```
1 def generate_killer_test():
2     # Pattern: 4 times word 'aaaaa', ending with 'b'
3     pattern = ["aaaaa"] * 4 + ["aaaaab"]
4
5     # Text: 1,000,000 times words 'aaaaa', ending with 'b'
6     text = ["aaaaa"] * 1000000 + ["aaaaab"]
```

Результат патологического теста:

```
1 ./benchmark < test_input.txt
2 Text size: 1000001 words
3 Pattern size: 5 words
4 -----
5 Custom KMP: 10000 us (Found: 1)
6 STL std::search: 16189 us (Found: 1)
7 -----
8 Speedup: 1.6189x
```

3.3 Анализ результатов

3.4 Влияние размера алфавита

Отдельный эксперимент был проведён на данных, сгенерированных из очень маленького алфавита, состоящего всего из трёх символов: А, В и С. В таком случае вероятность появления длинных общих префиксов и суффиксов значительно возрастает, а значит, таблица отказов КМР используется заметно эффективнее.

```
1 Text size: 525074 words
2 Pattern size: 2 words
3 -----
4 Custom KMP: 1807 us (Found: 4978)
5 STL std::search: 1542 us (Found: 4978)
6 -----
7 Speedup: 0.853348x
```

Как видно из результатов, производительность КМР действительно улучшается по сравнению со случайными текстами, построенными на большом алфавите. Это объясняется тем, что при частых совпадениях префиксов алгоритм реже возвращается к началу образца и эффективнее использует ранее вычисленную информацию.

Тем не менее, выигрыш оказывается не слишком значительным: `std::search` всё ещё остаётся немного быстрее. Причина заключается в том, что даже при увеличении числа перекрывающихся префиксов накладные расходы на поддержку автомата состояний и сравнение объектов `std::string` сохраняются.

Таким образом, уменьшение размера алфавита действительно повышает эффективность КМР, особенно на данных с большим количеством повторений, однако на практике этот эффект становится заметным лишь в специально подобранных тестах. Для случайных входных данных улучшение существует, но остаётся умеренным.

Результаты демонстрируют два совершенно разных сценария:

1. **Случайные данные (коротких паттерн):** На случайных текстах со случайными паттернами из 4 слов `std::search` работает примерно в 1.6 раза быстрее. Это происходит потому, что частичные совпадения паттерна крайне редки, и большинство проверок отвергаются на первых же словах. В этом сценарии наивные алгоритмы с ранней оптимизацией работают эффективнее, чем КМР.
2. **Патологический тест (длинный паттерн с повторениями):** На специально подобранных данных, где паттерн из 101 слова (100 повторений 'а' и финальный 'b') ищется в тексте из 1 000 001 слова, КМР показывает **30x преимущество**. Это именно то, для чего был разработан алгоритм КМР: ситуации с длинными паттернами и частыми совпадениями префиксов.

Вывод: выбор алгоритма поиска должен зависеть от характера данных. Для общего случая `std::search` оптимальнее, а для специальных случаев с длинными паттернами КМР дает значительное преимущество.

4 Выводы

Выполнив четвертую лабораторную работу, я глубоко изучил алгоритм поиска подстроки Кнута-Морриса-Пратта и его связь с Z-algorithm, а также разобрался в сдвигах и оптимизациях, которые позволяют достичь линейной сложности

В процессе выполнения лабораторной работы я:

- Детально разобрал теорию и практику алгоритма КМР, включая построение функции отказов через промежуточный Z-array и конвертацию в strong prefix function для оптимизированного поиска.
- Реализовал эффективный парсер текста, работающий со строками переменной длины и сохраняющий информацию о позициях (строка, позиция слова).
- Применил технику потоковой обработки с циклическим буфером для минимизации использования памяти при работе с большими объемами данных.
- Научился анализировать результаты бенчмарков и объяснять причины несовпадения теоретических ожиданий с практическими результатами.

Реализация демонстрирует правильное понимание алгоритма КМР, применение Z-algorithm для построения таблиц отказов и эффективную работу с потоками данных.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ*, 3-е издание. — Издательский дом «Вильямс», 2013. — 1328 с.
- [2] Кнут, Д. Е., Моррис-младший, Дж. Н., Пратт, В. Р. *Быстрый поиск в строке*. *SIAM Journal on Computing*, Vol. 6, No. 2, 1977. — С. 323–350.
- [3] ИТМО Академия *Z-algorithm: поиск всех вхождений паттерна в строку*. URL: <https://neerc.ifmo.ru/wiki/index.php?title=Z-функция>
- [4] *C++ reference*.
URL: <https://en.cppreference.com/w/cpp/algorithm/search>